PipeSense-ARM: A Lightweight Hardware Observer for Safe Adaptive Pipeline Reconfiguration in Embedded ARM-Like Processors

Author Name
Affiliation - email@example.com

Abstract- Embedded processors execute workloads whose bottlenecks change across short program phases: branch-heavy loops stress the front end, memory-heavy regions expose load latency, dependency-heavy code stresses bypass paths, and idle regions reward reduced activity. Static pipeline policy is simple, but it can be mismatched to these changing phases. This paper presents PipeSense-ARM, a SystemVerilog research prototype that places a lightweight hardware-resident observer and controller inside a small ARM-like educational five-stage pipeline. The observer samples narrow microarchitectural taps, classifies rolling windows into bottleneck phases, and requests one of several visible microarchitectural modes. A reconfiguration unit gates fetch and switches modes only after the pipeline reaches a safe boundary, bounding reconfiguration overhead and avoiding lost or duplicated instructions. Across ten embedded-style benchmarks and six execution configurations, adaptive PipeSense-ARM reduces cycles by up to 13.17% and improves IPC by up to 15.17% relative to a static-normal baseline. Directed runs match a sequential ISA reference model and report zero safety faults, while a 500-seed constrained-random safety run reports zero assertion failures, zero safety faults, and zero timeouts. Oracle fixed-mode comparisons show that the current controller remains conservative and does not outperform the best fixed mode chosen with hindsight. These results position PipeSense-ARM as a reviewable undergraduate architecture artifact and a foundation for richer adaptive-control experiments, not as a commercial ARM-compatible processor.

Index Terms- computer architecture, embedded processors, adaptive microarchitecture, pipeline reconfiguration, hardware performance monitoring

INTRODUCTION

Small embedded processors are attractive because they are predictable, inexpensive, and easy to integrate into low-power systems. Their simplicity also creates a tension: a single static pipeline policy may be acceptable on average but poorly matched to local program behavior. A branch-heavy control loop benefits from lower branch penalty, a memory-heavy loop benefits from latency mitigation, a dependency-heavy loop benefits from aggressive bypassing, and idle or low-utilization regions benefit from lower activity. PipeSense-ARM asks a narrow, artifact-driven question: can a tiny hardware observer inside a small ARM-like embedded pipeline classify these phases and safely reconfigure visible microarchitectural modes at runtime?

PipeSense-ARM implements a simplified ARM/RISC-style instruction set and a five-stage IF, ID, EX, MEM, WB pipeline with valid bits, load-use hazard detection, forwarding, branch flushes, memory wait modeling, writeback, and performance counters. Around that core, it adds three hardware-control blocks: a pipeline observer, a hysteretic adaptive controller, and a drain-before-switch reconfiguration unit. The design is intentionally sequential, closed loop, and microarchitecture-aware. It is not an ARM-compatible processor and not a commercial ARM implementation.

The contribution is not that phase detection, branch optimization, memory buffering, or low-power gating are individually new. Prior work has studied branch prediction [1981], memory prefetch and buffering [1990], program phase behavior [2002], reconfigurable microarchitectural structures [2002, 2000], and architectural power modeling [2000, 2001, 2003]. Instead, this paper contributes an integrated, inspectable hardware-control artifact: a small observer classifies bottleneck phases from live pipeline taps, a controller maps phases to modes with residency constraints, and a reconfiguration unit enforces safe switching.

This paper makes three claims. First, the design is complete enough to behave like a sequential microarchitectural prototype rather than a paper-only concept. Second, the evaluation shows that adaptive control can improve over a static-normal baseline on phase-biased synthetic workloads. Third, the oracle comparison reveals remaining controller limitations, especially on workloads where memory mitigation dominates and a best fixed mode chosen with hindsight is much stronger than the current adaptive policy.

BACKGROUND AND RELATED WORK

Computer architecture evaluation commonly separates mechanism, policy, and workload. A mechanism provides a capability, such as branch prediction, forwarding, or memory prefetching. A policy decides when and how the mechanism should be used. A workload determines whether the policy is useful. PipeSense-ARM follows this separation by exposing simple mechanisms as visible modes, then asking whether a lightweight hardware policy can select among them from pipeline observations.

The design draws from several research threads while keeping its claim boundary explicit. Smith's branch-prediction study established that control-flow behavior can be measured and exploited by hardware policy [1981]; PipeSense-ARM uses only an early-resolution toy branch mode, not a realistic predictor. Jouppi's victim-cache and prefetch-buffer work and later reconfigurable memory-hierarchy work show that small memory structures can trade performance and energy [1990, 2000]; PipeSense-ARM uses synthetic wait mitigation, not a cache or true prefetcher. Sherwood et al. and Dhodapkar and Smith showed that phase behavior can guide simulation and multi-configuration hardware [2002, 2002]; PipeSense-ARM uses short in-core event windows rather than signatures or long traces.

The measurement and safety framing is similarly scoped. Wattch, Mudge, and Isci and Martonosi motivate architectural power and runtime counter evidence [2000, 2001, 2003], but this paper reports an activity proxy, not physical energy. SystemVerilog Assertions, assertion-based design, model checking, and reconfigurable-system verification provide methodological background for checking runtime control [2023, 2004, 1986, 2007]; PipeSense-ARM provides simulation monitors and bounded abstract formal scaffolding, not formal verification of the full RTL core. Finally, the artifact follows the executable-modeling tradition associated with SimpleScalar and quantitative computer architecture methodology [2002, 2017]: claims should be tied

to code, baselines, and measurements.

PIPESENSE-ARM ARCHITECTURE

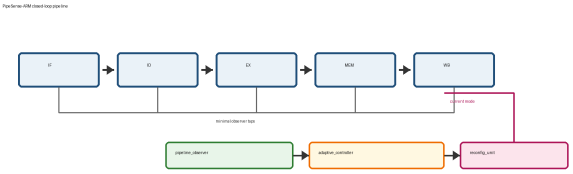

Fig. 1. PipeSense-ARM places an observer/controller/reconfiguration loop around a small five-stage pipeline.

Fig. the corresponding table or figure summarizes the architecture. The baseline core is a five-stage educational pipeline. Instructions are 32 bits wide with a 4-bit opcode, three 4-bit register fields or branch-condition field, and a 16-bit immediate or branch target. The supported operations are ADD, SUB, AND, ORR, EOR, LDR, STR, B, CMP, NOP, and a simulation-only HALT sentinel. The design includes a program counter, register file, instruction memory, data memory, pipeline registers, valid bits, stall and flush signals, forwarding, load-use detection, branch handling, memory wait modeling, and performance counters.

The observer reads minimal taps: stage-valid bits, IF/ID/EX stall signals, branch flushes, branch-taken events, load-use hazards, memory waits, retirements, and cycle count. Over a parameterized rolling window, it classifies observed behavior into six phase labels: balanced, branch-heavy, memory-stall, load-use-hazard, front-end-stall, or idle/low-utilization. The first prototype uses threshold logic rather than a learned classifier because counters, comparators, and fixed thresholds are easier to inspect and synthesize.

The adaptive controller maps phase labels to five modes. MODE_NORMAL uses default branch, forwarding, and memory behavior. MODE_BRANCH_OPT reduces the modeled branch penalty through earlier branch handling. MODE_MEMORY_OPT suppresses deterministic synthetic memory waits to represent a small prefetch or buffering effect. MODE_HAZARD_OPT enables a simplified aggressive load-result bypass path. MODE_LOW_POWER reduces the activity-energy proxy during low-utilization execution. The controller includes stable-phase hysteresis and a minimum mode residency counter so that phase noise does not cause mode thrashing.

Each mode changes a measurable path in the pipeline model. The branch mode is not a full branch predictor; it has no branch target buffer, saturating counters, or speculative recovery machinery. The memory mode is not a real cache hierarchy; a physical design would need storage, tags, ordering rules, and prefetch policy. The hazard mode represents an extra load-result bypass path and forces the checks to cover same-cycle writeback-to-decode behavior. The low-power mode changes an activity proxy rather than clock-tree or power-gate cells. These abstractions keep the prototype small while preserving a visible connection between observed bottlenecks and controlled hardware behavior.

The control loop is deliberately placed at the boundary between local pipeline timing and architectural state. The observer does not inspect program source, benchmark labels, or software hints. It only sees events that real pipeline hardware could expose as counter increments or one-cycle pulses. The controller therefore makes decisions with stale and lossy information: a phase must last long enough to fill the observation window, satisfy stable-phase hysteresis, and then survive the reconfiguration drain. This timing is part of the research question rather than an implementation accident. A policy that works only with perfect hindsight would be captured by the oracle baseline, not by the adaptive hardware.

The design also separates mode request from mode commit. A requested mode may be visible inside the controller before it becomes the current mode seen by the pipeline. During that interval, the reconfiguration unit gates fetch and waits for the safe boundary. This distinction makes the logs easier to interpret: reconfiguration count measures committed switches, while reconfiguration penalty counts cycles spent draining and gating. It also avoids a subtle accounting problem in which a benchmark appears to enter a mode even though no instruction actually executes under that mode.

SAFE RECONFIGURATION AND IMPLEMENTATION

The reconfiguration unit receives a requested mode and the current pipeline-empty and memory-wait status. If the requested mode differs from the current mode, it asserts a reconfiguration-active state and gates fetch. It then waits until IF/ID, ID/EX, EX/MEM, and MEM/WB valid bits are clear and no memory wait is active. Only at that safe boundary does it install the requested mode, pulse reconfiguration done, and update reconfiguration counters. The protocol is conservative but transparent: it trades extra cycles for a simple correctness argument.

The testbench strengthens this argument with simulation-time safety evidence. Instructions are tagged as they move through the pipeline. Monitors flag illegal mode changes outside empty-pipeline boundaries, reconfiguration completion before drain, fetch not gated during active reconfiguration, and duplicate or nonmonotonic retirement tags. The reported safety_faults field must be zero before any result is considered usable. A separate 500-seed constrained-random safety regression produces 2,500 mode-result rows with zero assertion failures, zero safety faults, and zero timeouts. A sequential ISA reference model also compares retired instruction counts and final architectural data-state hashes against the HDL output. The repository includes an abstract formal token-conservation harness for fetch, stall, flush, and retirement behavior, but this is not formal verification of the full processor.

The RTL is organized into small SystemVerilog modules. arm_like_core.sv integrates the pipeline and mode behavior. pipeline_observer.sv implements rolling counters and phase classification. adaptive_controller.sv implements hysteresis, residency tracking, and mode requests. reconfig_unit.sv implements safe switching and reconfiguration accounting. Separate hazard, forwarding, memory, counter, and definition modules keep the design inspectable. The core also handles writeback-to-decode bypassing after load stalls, fresh condition-code visibility for CMP-to-branch sequences, and held EX-stage operands during memory waits.

Drain-before-switch is intentionally conservative. It prevents a branch mode change from modifying flush behavior while an older branch is in flight, prevents a hazard mode change from altering forwarding assumptions for a dependent instruction, and prevents a memory mode change from changing wait behavior around an outstanding memory operation. The cost is visible in the results: a mode can improve local execution but still lose if the useful phase is short and the drain cost is high. That is why the paper reports reconfiguration count and reconfiguration penalty beside IPC and cycles.

The Python scripts are part of the artifact. run_sim.py compiles and runs the testbench with Icarus Verilog, parses results, validates CSVs, runs the ISA reference model, and compares HDL output to the reference. analyze_results.py computes IPC, CPI, normalized event rates, adaptive improvement over static normal, and the oracle fixed-mode gap. run_sweep.py runs observer-window, threshold-profile, and residency sweeps. check_artifact.py runs repository, syntax, benchmark-parity, fixture, reference-model, paper, and preview guardrails.

EVALUATION METHODOLOGY

The evaluation uses ten embedded-style benchmarks: arithmetic-heavy, branch-heavy, memory-heavy, load-use-heavy, mixed-control, tiny-FIR, dhrystone-toy, coremark-toy, dsp-fir-codegen, and pid-control-codegen. The first six programs are intentionally small and phase-biased. The Dhrystone-style and CoreMark-style programs are toy-ISA ports inspired by recognizable integer/control and pointer/checksum patterns. The two generated-style kernels mimic straight-line compiler output for a small finite-impulse-response loop and a proportional-integral embedded-control loop. None are full benchmark-suite ports, but they make the workload set less anonymous while preserving instruction-for-instruction comparability with the sequential reference model.

Each benchmark is executed in six configurations: static normal, fixed branch, fixed memory, fixed hazard, fixed low-power, and adaptive PipeSense-ARM. The fixed modes provide mechanism-level baselines. Static normal asks whether adaptation improves over a simple nonadaptive design. The best fixed mode chosen with hindsight serves as an oracle baseline. That oracle is not deployable, but it is important: if adaptive mode beats static normal but is far worse than the oracle, the result should be interpreted as controller headroom rather than a final performance victory.

Metrics include total cycles, retired instructions, IPC, CPI, stall cycles, flush cycles, memory wait cycles, load-use stalls, reconfiguration count, reconfiguration penalty, an activity-energy proxy, safety faults, timeout status, and final architectural data-state hash. The activity-energy proxy is not physical power. It is an RTL-level activity estimate used to compare relative mode behavior before gate-level toggle data are available. Hardware cost is reported with a Yosys generic-cell proxy and older analytical notes; neither is calibrated timing, power, FPGA utilization, or physical area evidence.

The evaluation order is part of the methodology. Simulation results are not cited until the CSV validator passes, the ISA reference comparison passes, and the paper checker confirms that manuscript tables match generated files. The sweep and fuzz regressions are separate evidence paths: the sweep asks whether the controller remains analyzable across window, threshold, and residency choices, while the fuzz regression stresses safety properties under randomized program and mode interactions. This ordering reduces the risk that a favorable table survives after the underlying RTL or benchmark model has changed.

RESULTS

The current validated HDL run contains 60 benchmark/mode rows. Every run halted, no run timed out, and every run reported zero safety faults. HDL retired counts and final architectural data-state hashes match the sequential ISA reference model. The 500-seed safety regression adds randomized stress evidence and hits several reconfiguration-interaction buckets, including hazard-during-reconfiguration, back-to-back reconfiguration requests, and reconfiguration-then-branch. Table the corresponding table or figure compares adaptive PipeSense-ARM against static normal mode.

Adaptive control improves over static normal mode on four of the ten benchmarks. The largest cycle reduction is 13.17% on load-use-heavy code, where the observer moves toward the hazard-optimized path and reduces load-use stalls. Branch-heavy code improves by 10.94% in cycles and 12.28% in IPC. Memory-heavy code improves by 12.99% in cycles and shows the largest activity-energy proxy reduction, 15.68%, but it pays six reconfigurations and 28 reconfiguration penalty cycles. Tiny-FIR, dsp-fir-codegen, and pid-control-codegen are useful negative cases: adaptive mode slightly increases cycles, showing that mode switching can cost more than it saves on short mixed workloads.

Across all ten adaptive runs, static-normal execution takes 1,563 cycles and adaptive execution takes 1,499 cycles, an aggregate cycle reduction of 4.09%. The adaptive activity-energy proxy falls from 8,872 to 8,508, an aggregate reduction of 4.10%. Averaged per benchmark, the cycle reduction is 3.43%, IPC improvement is 3.99%, and activity-energy proxy reduction is 3.50%. These aggregate numbers are intentionally reported beside the per-benchmark rows because the averages hide the controller misses on dsp-fir-codegen, pid-control-codegen, and tiny-FIR.

Table the corresponding table or figure compares adaptive mode with the best fixed mode for each benchmark. Negative gaps mean adaptive is worse than an oracle fixed mode chosen after seeing the workload.

The oracle comparison is the most important result for claim discipline. Adaptive PipeSense-ARM is better than static normal mode on several phase-biased tests, but it does not beat the best fixed mode. The largest gaps occur when fixed memory mode dominates. For memory-heavy code, fixed memory mode completes in 147 cycles while adaptive mode takes 201 cycles. That result should be presented as controller headroom: the current threshold and residency policy does not move into memory-optimized mode early enough or remain there long enough.

The average adaptive gap to the best fixed mode is -14.41% in cycles and -8.43% in the activity-energy proxy. The negative sign is important: the oracle is not a baseline that adaptive beats, but a hindsight upper bound for these fixed mechanisms. This makes the result less flattering and more useful. It shows that the mode mechanisms themselves can help substantially, while the current online controller is still too conservative or too late on several workloads.

Table the corresponding table or figure isolates three design assumptions. Disabling the observer or disabling the controller removes all adaptive switching, so the adaptive run falls back to the static-normal aggregate of 1,563 cycles. The zero-cost row is an analytical idealization computed from the validated full-adaptive run by subtracting reconfiguration penalty cycles; it is not an unsafe instant-switch RTL run. It estimates that eliminating drain cost would improve the default adaptive total from 1,499 to 1,426 cycles, a 4.87% headroom bound.

Table the corresponding table or figure reports the current Yosys generic-cell area proxy. The baseline core proxy contains 1,830 generic cells. Synthesizing the observer, controller, and reconfiguration modules as standalone proxy modules gives 2,819 generic cells, or 154.04% of the baseline core proxy. These are not synthesis results for the full SystemVerilog RTL, and they are not calibrated to an FPGA family, standard-cell library, timing target, or power model.

Table I. Adaptive PipeSense versus static-normal baseline.

Benchmark	Norm	Adapt	Cycle	IPC	Energy	Rcfg
arithmetic heavy	30	30	0.00%	0.00%	0.00%	0
branch heavy	128	114	10.94%	12.28%	6.64%	1
coremark toy	162	162	0.00%	0.00%	0.00%	0
dhrystone toy	133	133	0.00%	0.00%	0.00%	0
dsp fir codegen	192	194	-1.04%	-1.03%	0.36%	2
load use heavy	205	178	13.17%	15.17%	7.64%	1
memory heavy	231	201	12.99%	14.92%	15.68%	6
mixed control	131	127	3.05%	3.14%	2.28%	1
pid control codegen	192	200	-4.17%	-3.99%	-0.79%	3
tiny fir	159	160	-0.63%	-0.62%	3.17%	3

Table II. Adaptive mode versus best fixed-mode oracle.

Benchmark	Best fixed	Best	Adapt	Cycle gap	Energy gap
arithmetic heavy	hazard	29	30	-3.45%	-1.73%
branch heavy	branch	105	114	-8.57%	-3.94%
coremark toy	hazard	150	162	-8.00%	-3.24%
dhrystone toy	branch	124	133	-7.26%	-3.38%
dsp fir codegen	hazard	168	194	-15.48%	-5.35%
load use heavy	hazard	169	178	-5.33%	-2.16%
memory heavy	memory	147	201	-36.73%	-23.72%
mixed control	memory	105	127	-20.95%	-18.31%
pid control codegen	memory	178	200	-12.36%	-6.76%
tiny fir	memory	127	160	-25.98%	-15.70%

Table III. Analytical hardware-cost estimate, not synthesis evidence.

Component	FF bits	Comp.	Adders
pipeline observer	228	5	7
adaptive controller	31	4	2
reconfig unit	104	2	3
safety monitor sim	193	4	1

PARAMETER SENSITIVITY

The repository includes a 3x3x3 sweep over observer window size, minimum mode residency, and threshold profile. The tested windows are 16, 32, and 64 cycles; the tested residencies are 8, 24, and 48 cycles; and the tested threshold profiles are tight, medium, and loose. All 27 settings complete with 60 benchmark/mode rows and zero validation failures. Aggregate adaptive cycles range from 1,477 cycles with a 64-cycle window and loose thresholds to 1,670 cycles with a 16-cycle window, 8-cycle residency, and tight thresholds. The default setting sits near the low end at 1,499 cycles and 17 reconfigurations. The sweep records 318 adaptive cycle wins and 1,032 non-wins across adaptive-versus-fixed cells, reinforcing that the controller is sensitive to phase-classification policy.

The sweep validates that the artifact can explore sensitivity, but it does not prove robustness for general embedded software. The benchmarks are short and phase-biased, so parameter changes can swing transition counts. A richer workload suite should include phase changes near the classification thresholds, randomized latency behavior, and longer mixed workloads where window size and residency should matter.

DISCUSSION AND LIMITATIONS

The results support a modest claim. A small hardware-resident observer can detect phase-biased bottlenecks and guide safe adaptive mode switching in a sequential pipeline prototype. The design includes hazards, forwarding, stalls, flushes, memory waits, valid bits, counters, writeback timing details, and reconfiguration safety checks. It improves over static normal mode on

several tests, but the oracle comparison shows that a well-chosen fixed mode can be substantially better on these short synthetic workloads. This is not a failure to hide; it identifies the next controller problems.

The main threats to validity are workload representativeness, synthetic memory behavior, proxy energy, and safety coverage. The ten benchmarks are small, and most are structured to expose specific bottlenecks. The dhrystone-toy, coremark-toy, dsp-fir-codegen, and pid-control-codegen additions improve recognizability, but they are still toy-ISA kernels rather than full compiler-generated benchmark ports. The memory wait behavior is deterministic and address based, not a cache, bus, scratchpad, DMA, or DRAM model. The activity-energy proxy is not physical energy. The simulation monitors, architectural hashes, fuzz regression, and abstract token harness are stronger than prose alone, but they are not a full formal proof of the processor.

A second limitation is that the modes are intentionally stylized. MODE_BRANCH_OPT stands in for earlier branch handling, MODE_MEMORY_OPT stands in for memory-wait mitigation, MODE_HAZARD_OPT stands in for a stronger bypass path, and MODE_LOW_POWER stands in for reduced switching activity. This is appropriate for isolating the control loop, but it means the paper should not claim a ready processor optimization. A stronger architecture study would replace each knob with a realistic hardware mechanism, then rerun the same static, adaptive, oracle, sweep, fuzz, and area checks.

The area result is also intentionally cautious. The Yosys flow reports a generic-cell proxy for standalone observer, controller, and reconfiguration modules and compares their cell counts to a baseline core proxy. It does not close timing, account for integration wiring, map to FPGA LUTs, estimate clock frequency, or report physical power. The large 154.04% proxy overhead should therefore be read as an early warning about prototype cost, not as calibrated silicon evidence.

Future work should replace the synthetic memory knob with a small cache, scratchpad, or stream buffer; tune the controller with mode-specific entry and exit thresholds; lower reconfiguration overhead without weakening safety; add full compiler-generated embedded benchmark ports; and run synthesis on a named FPGA or standard-cell target. Formal properties should be bound directly into the RTL core for drain, fetch gating, mode commit, and retirement invariants.

ARTIFACT AND REPRODUCIBILITY

The repository is organized so that paper claims can be checked directly. The RTL directory contains the core, observer, controller, reconfiguration unit, hazard logic, forwarding logic, memories, counters, and shared definitions. The testbench directory contains the benchmark programs and simulation harness. The scripts directory contains simulation, analysis, validation, plotting, parameter sweep, reference-model, paper-checking, and preview scripts. The docs directory records methodology, threats to validity, hardware-realism notes, reproducibility notes, and requirements traceability.

The primary command is python scripts/run_sim.py. With Icarus Verilog installed, it compiles the SystemVerilog testbench, runs all 60 benchmark/mode combinations, writes the raw simulation log, generates result CSVs, validates the CSV fields, runs the sequential ISA reference model, and compares HDL output against reference retired counts and architectural hashes. python scripts/check_paper.py verifies that the manuscript has no unresolved placeholder tokens, that every cited bibliography key exists in references.bib, that claim-discipline language is present, and that adaptive, oracle, ablation, and area result tables match generated CSVs when those CSVs are available. python scripts/build_paper_preview.py and python scripts/verify_paper_preview.py generate and check a six-page preview PDF without requiring LaTeX.

Generated CSVs, simulator logs, rendered pages, and build products are intentionally ignored so the repository does not accumulate bulky run artifacts. The checked-in visibility point is results/SUMMARY.md, which records the scripts that ran cleanly, the headline result numbers, the safety fault count, the area-proxy status, paper-check status, and remaining blockers. This keeps GitHub readable while still forcing raw data to be regenerated locally before numbers are changed.

The artifact still needs final submission hygiene. On a machine with pdflatex and bibtex, the manuscript can be built from paper/pipesense_urtc_8page.tex. The author block must be replaced before submission, and current venue rules should be checked. This source is now a six-page workshop draft; if a stricter page cap applies, the safest additional cuts are discussion, future work, and artifact prose, not the architecture description or result tables.

CONCLUSION

PipeSense-ARM demonstrates a lightweight hardware-control loop for safe adaptive pipeline reconfiguration in a small ARM-like educational processor. The prototype combines a five-stage pipeline, narrow pipeline-event observation, threshold phase classification, hysteretic mode selection, and conservative drain-before-switch reconfiguration. Validated simulations show that adaptive mode improves over static normal execution on several phase-biased workloads while preserving zero safety faults and matching a sequential reference model. The stricter oracle comparison shows that the current controller remains far from the best fixed mode on memory-dominated tests, which bounds the claim and motivates future work. Overall, PipeSense-ARM is best framed as a research artifact and evaluation scaffold for hardware-native adaptive processor control.

REFERENCES

[1] J. E. Smith, A Study of Branch Prediction Strategies, ISCA, 1981.

[2] N. P. Jouppi, Improving Direct-Mapped Cache Performance by the Addition of a Small Fully-Associative Cache and Prefetch Buffers, ISCA, 1990.

[3] T. Sherwood, E. Perelman, G. Hamerly, and B. Calder, Automatically Characterizing Large Scale Program Behavior, ASPLOS, 2002.

[4] A. S. Dhodapkar and J. E. Smith, Managing Multi-Configuration Hardware via Dynamic Working Set Analysis, ISCA, 2002.

[5] R. Balasubramonian, D. H. Albonesi, A. Buyuktosunoglu, and S. Dwarkadas, Memory Hierarchy Reconfiguration for Energy and Performance, MICRO, 2000.

[6] D. Brooks, V. Tiwari, and M. Martonosi, Wattch: A Framework for Architectural-Level Power Analysis and Optimizations, ISCA, 2000.

[7] T. Mudge, Power: A First-Class Architectural Design Constraint, Computer, 2001.

[8] C. Isci and M. Martonosi, Runtime Power Monitoring in High-End Processors, MICRO, 2003.

[9] T. Austin, E. Larson, and D. Ernst, SimpleScalar: An Infrastructure for Computer System Modeling, Computer, 2002.

[10] J. L. Hennessy and D. A. Patterson, Computer Architecture: A Quantitative Approach, 6th ed., 2017.

[11] IEEE, IEEE Standard for SystemVerilog--Unified Hardware Design, Specification, and Verification Language, IEEE Std 1800-2023, 2023.

[12] H. D. Foster, A. C. Krolnik, and D. J. Lacey, Assertion-Based Design, Kluwer Academic Publishers, 2004.
[13] E. M. Clarke, E. A. Emerson, and A. P. Sistla, Automatic Verification of Finite-State Concurrent Systems Using Temporal Logic Specifications, TOPLAS, 1986.
[14] M. Borgatti et al., An Integrated Design and Verification Methodology for Reconfigurable Multimedia Systems, arXiv:0710.4846, 2007.